

Advanced Creative Module Enhancements

Asset Pipeline Injection (Texture & Motion in Design Tokens)

Currently, **Design OS** uses the `/design-tokens` step to capture colors and typography for the design system (stored in `product/design-system/colors.json` and `typography.json` ¹ ²). To inject a new **Texture & Motion** asset pipeline, we can extend this step to handle an additional definition file (e.g. `product/design-system/motion.json`). This involves:

- **Adding a Motion Definition File:** Define a new JSON schema for motion/physics parameters (e.g. animation durations, easing curves, 3D texture settings) and create `motion.json` under the design-system folder alongside colors and typography ¹. This file will store key “texture & motion” tokens (like particle effects, transition speeds, etc.).
- **Modifying the `/design-tokens` Workflow:** Update the design-tokens phase to prompt for motion/animation style preferences after color palette and fonts are chosen. For example, once Tailwind colors and Google Fonts are set, the agent should ask the user about global motion design (e.g. “Describe the app’s animation style or physics (snappy, bouncy, smooth, etc.)”). The agent will then generate `motion.json` with the specified values ², similar to how it generates the other token files.
- **Integrating Motion Tokens:** Ensure the application shell and section components load and apply these motion tokens. For instance, the `AppShell` or relevant components can read from `motion.json` to apply consistent animation curves and texture effects globally. By treating motion parameters as first-class design tokens, the exported handoff package will include them for the implementation agent. This means extending the export logic (see below) to include the new `motion.json` in the assets bundle ³.

Overall, this change injects a “Texture & Motion” layer into the design system phase, guaranteeing that dynamic visual behaviors (like WebGL shaders or CSS animations) are defined upfront and included in the design spec.

Component Enhancement (Creative Brief Field in Section Specs)

Design OS currently defines each **Section** via a spec markdown (`spec.md`) generated by the `/shape-section` command. This spec includes an overview, user flows, UI requirements, and scope boundaries ⁴, but no explicit field for a high-level creative direction. To force the agent to define a “**Visual Hook**” for each section before coding, we can inject a **Creative Brief** field into these section specs:

- **Extending Section Spec Format:** Modify the content template of `product/sections/[section]/spec.md` to include a **Creative Brief** section. For example, after the “Scope” section, add a heading like **Creative Brief:** followed by a short markdown description of the unique visual concept for the section. This brief should describe the signature visual element or

theme (e.g. “This section uses a WebGL-based distortion effect for its hero image to create a dynamic, interactive feel.”).

- **Updating** `/shape-section` **Prompt:** Adjust the agent’s conversation in the section-shaping step to explicitly ask for this creative brief. After gathering flows and requirements, the AI should prompt the user to articulate the visual/aesthetic hook of the section. This ensures the AI (Design OS agent) formulates a clear *visual design intention* in writing before implementation. The interactive flow might be: “Now describe the visual theme or hook for this section (e.g. any special animations, illustrations, or style inspiration)”. The agent would then incorporate the answer into the `spec.md` under the **Creative Brief** field.
- **Enforcing the Field:** To **force** the agent to include a visual hook, we can add a rule or check so that the spec is not considered complete until a non-empty Creative Brief is present. Practically, this could be done by validating the `spec.md` content structure or by having the agent confirm: “*Visual hook defined* .” The presence of this field in the spec guides the subsequent `/design-screen` step, reminding the agent to implement that unique visual element during UI coding.

By injecting a **Creative Brief** into section designs, every section starts with a defined creative vision. This helps the implementation agent maintain stylistic cohesion and ensures that each section’s UI has a standout visual feature as part of its requirements ⁴.

Export Logic (Including “Nano Banana” Video Assets in Handoff)

The `/export-product` command packages all design outputs into a handoff bundle, including design tokens, models, shell specs, section specs, data, and images ³. To ensure “Nano Banana” video assets are not missed by the implementation agent, we need to modify the export logic to include these files:

- **Identify Asset Location:** First, determine where Nano Banana outputs its video files. If, for example, Nano Banana is used to generate UI placeholder videos or animated assets, these might be saved in a specific folder (perhaps a subfolder of `product/sections/[section-id]/` or a global `assets/` directory). Let’s assume they reside alongside section screenshots or in a known asset path.
- **Include Video Files in Export:** Update the export script to capture these video assets. In the export generation code, where it currently copies portable assets (design-system files, `shell/`, `sections/` content, etc.), add logic to include video files (e.g. `.mp4`, `.webm`, or similar). For instance, if Nano Banana videos are stored as `product/sections/[section]/video.mp4`, ensure that directory is included when copying `sections/` into the `product-plan` output ³. This might mean adjusting file filters to not just grab `*.png` screenshots but also any media files.
- **Package Verification:** After modification, the exported `product-plan` should contain the Nano Banana videos in the appropriate location (perhaps under `product-plan/sections/[section]/` or a dedicated media folder). We should verify that the **README or instructions** in the handoff mention these assets so the implementation agent knows they exist. For example, add a line in the export’s README listing the video files and their intended use. This prevents the coding agent from overlooking the videos during development.

In summary, by tweaking the export logic to bundle Nano Banana’s video outputs, we preserve all creative assets in the final handoff. This guarantees the implementation agent receives the full set of visuals (including motion graphics or video loops) along with the code, thus maintaining the fidelity of the designer’s intent.

Custom Agent Directive (Anti-Slop Check Before Finalizing Components)

To uphold quality and avoid AI-induced “slop” in the final designs, we can add a new rule in **agents.md** that mandates an **Anti-Slop** review before any component is finalized. The **Anti-Slop list** typically refers to a checklist of unwanted patterns or artifacts (like meaningless filler text, overly generic styles, or redundant code/comment clutter) that should be “ruthlessly stripped” from AI outputs ⁵. The new directive could be phrased as follows:

- **Anti-Slop Review:** *“Before finalizing any section design or component, the Design OS agent must perform an Anti-Slop audit. Remove all generic or placeholder content and any AI-generated artifacts that reduce quality ⁵. This includes purging dummy text (e.g. lorem ipsum), overly safe/default design choices, extraneous comments, and sloppy code. Only finalize the spec or component after it passes this Anti-Slop check.”*

By adding this rule to `agents.md`, we formally require the AI agent to self-censor and refine its output against a quality benchmark. In practice, after completing a design or component code, the agent would revisit the “Anti-Slop” list and sanitize the output: e.g. replace boilerplate copy with meaningful text, ensure the visual design isn’t a cookie-cutter template, and clean up any unnecessary elements. This step leverages the existing concept of *Anti-Slop sanitization* (removing AI artifacts and noise ⁵) and applies it to the Design OS workflow. The result is a cleaner, more polished handoff package.

Finally, this new **agents.md** directive will act as a safeguard: it forces the Design OS agent to double-check its work for sloppiness every time, leading to higher-quality designs with minimal “AI slop.” The implementation agent will then start from a more refined baseline, requiring less cleanup downstream.

Sources: The repository structure and documentation for Design OS were referenced to identify where these enhancements apply. Key files and docs include the Design OS technical overview ¹ ³, section design workflow docs ⁴, and external guidelines on anti-slop practices ⁵. These inform the proposed modifications above.

¹ ² ³ [design-os-overview.md](#)

<https://github.com/quochuydev/quochuydev/blob/0d0f50ee8b61ce806622579b97d10a53c47614ba/ai/design-os-overview.md>

⁴ [Design OS Section Design](#)

<https://buildermethods.com/design-os/design-sections>

⁵ [Hona's gists · GitHub](#)

<https://gist.github.com/Hona>